Lab 5: Static Code Analysis

Pylint, Bandit, and Flake8 — Helpdesk Ticket Management System

Name: Rekha Dhorigol
SRN: PES1UG24CS370
Section: G

Repository: https://github.com/rekhadhorigol/SE-Labs-PES1UG24CS370

(Lab-5 folder)

1. Introduction

Static code analysis examines source code without running it to identify potential code-quality, security, maintainability & style problems. In this lab, the helpdesk ticket management program was analyzed using Pylint, Bandit, and Flake8. The reports were reviewed, issues were corrected & the tools were run again to verify the changes.

2. Static Analysis Tools

- **Pylint:** Evaluates Python code quality and programming practices. It reports issues such as missing documentation, naming problems, unused variables, and risky or unclear constructs.
- **Bandit:** Checks Python source code for common security risks, including hardcoded secrets and potentially unsafe subprocess usage.
- **Flake8:** Checks Python code for style and formatting problems, including whitespace, line length, exception syntax, and PEP 8 conventions.

3. Issues Identified and Fixes Applied

The table summarizes representative issues from the original reports & how the code was changed.

Issue	Type	Description	Fix / status
Mutable default argument	Correctness / maintainability	The original function used a list as a default argument, which can be shared across calls.	Changed the default to None and created a new list inside the function when no audit trail is provided.
Broad exception handling	Correctness / maintainability	A bare except can hide unexpected errors and make debugging difficult.	Caught the specific KeyError expected when a ticket ID is missing.
Hardcoded administrator password	Security	A password was embedded directly in the source code.	Read the password from the HELPDESK_ADMIN_PASSWORD environment variable; closing a ticket requires the variable to be configured and the supplied password to match.
Shell execution in subprocess	Security	The original ping call used shell=True, which can expose command execution to injection risks.	Passed arguments as a list and set shell=False.
Bandit subprocess findings remain	Security / tool finding	Bandit still reports low-severity findings for importing subprocess and using a subprocess call with the ping executable.	Kept the ping feature using a list of arguments and shell=False. These remaining findings are disclosed rather than claimed as eliminated; review whether ping functionality is required and whether hostname input should be further restricted.
Missing module/function docstrings	Code quality	The original module and functions lacked documentation.	Added a module docstring and function docstrings.

Naming and global-state warnings	Code quality	Pylint flagged the global statement and the ticket_counter naming convention.	The implementation retains global ticket state and ticket_counter; Pylint reports these as remaining warnings.
Old string formatting	Code quality	The original code used older formatting practices in places.	Updated affected strings to f-strings.
Dictionary key iteration	Code quality	The original loop iterated through dictionary keys when key-value pairs were needed.	Used items() to access ticket IDs and ticket data together.
Comparison with None	Code quality	The original code compared a value with None using equality.	Used the identity check 'is None'.
File handling and encoding	Reliability	The original JSON file handling did not explicitly specify an encoding and context-managed file access.	Used a with statement and encoding='utf-8' when writing the JSON file.
Flake8 formatting issues	Style	The original file had long lines, spacing/blank-line issues, a bare except, a None comparison issue, and an unused variable.	Reformatted lines and spacing, corrected exception handling and None comparison, and removed the unused variable.

4. Before-and-After Results

4.1 Pylint - Before

File: reports/pylint_before.txt
Original score: 5.53/10

pylint_before.txt

reports > pylint_before.txt

```
************* Module helpdesk_system
helpdesk_system.py:13:0: C0301: Line too long (118/100) (line-too-long)
helpdesk_system.py:1:0: C0114: Missing module docstring (missing-module-docstring)
helpdesk_system.py:7:0: C0103: Constant name "ticket_counter" doesn't conform to UPPER_CASE naming style (invalid-name)
helpdesk_system.py:10:0: C0116: Missing function or method docstring (missing-function-docstring)
helpdesk_system.py:10:0: W0102: Dangerous default value [] as argument (dangerous-default-value)
helpdesk_system.py:11:4: W0603: Using the global statement (global-statement)
helpdesk_system.py:14:23: C0209: Formatting a regular string which could be an f-string (consider-using-f-string)
helpdesk_system.py:17:0: C0116: Missing function or method docstring (missing-function-docstring)
helpdesk_system.py:20:0: C0116: Missing function or method docstring (missing-function-docstring)
helpdesk_system.py:23:4: W0702: No exception type(s) specified (bare-except)
helpdesk_system.py:26:0: C0116: Missing function or method docstring (missing-function-docstring)
helpdesk_system.py:30:0: C0116: Missing function or method docstring (missing-function-docstring)
helpdesk_system.py:30:0: C0103: Function name "unresolvedHighPriority" doesn't conform to snake_case naming style (invalid-name)
helpdesk_system.py:32:4: C0206: Consider iterating with .items() (consider-using-dict-items)
helpdesk_system.py:33:43: C0121: Comparison 'tickets[t]['tech'] == None' should be 'tickets[t]['tech'] is None' (singleton-comparison)
helpdesk_system.py:37:0: C0116: Missing function or method docstring (missing-function-docstring)
helpdesk_system.py:40:0: C0116: Missing function or method docstring (missing-function-docstring)
helpdesk_system.py:41:8: W1514: Using open without explicitly specifying an encoding (unspecified-encoding)
helpdesk_system.py:41:8: R1732: Consider using 'with' for resource-allocating operations (consider-using-with)
helpdesk_system.py:45:0: C0116: Missing function or method docstring (missing-function-docstring)
helpdesk_system.py:47:4: W0612: Unused variable 't2' (unused-variable)

-----------------------------------
Your code has been rated at 5.53/10
```

4.2 Pylint - After

File: reports/pylint_after_pylint.txt
Final score: 9.67/10. Two warnings remain: the global statement and ticket_counter naming convention.

pylint_after_pylint.txt

reports > pylint_after_pylint.txt

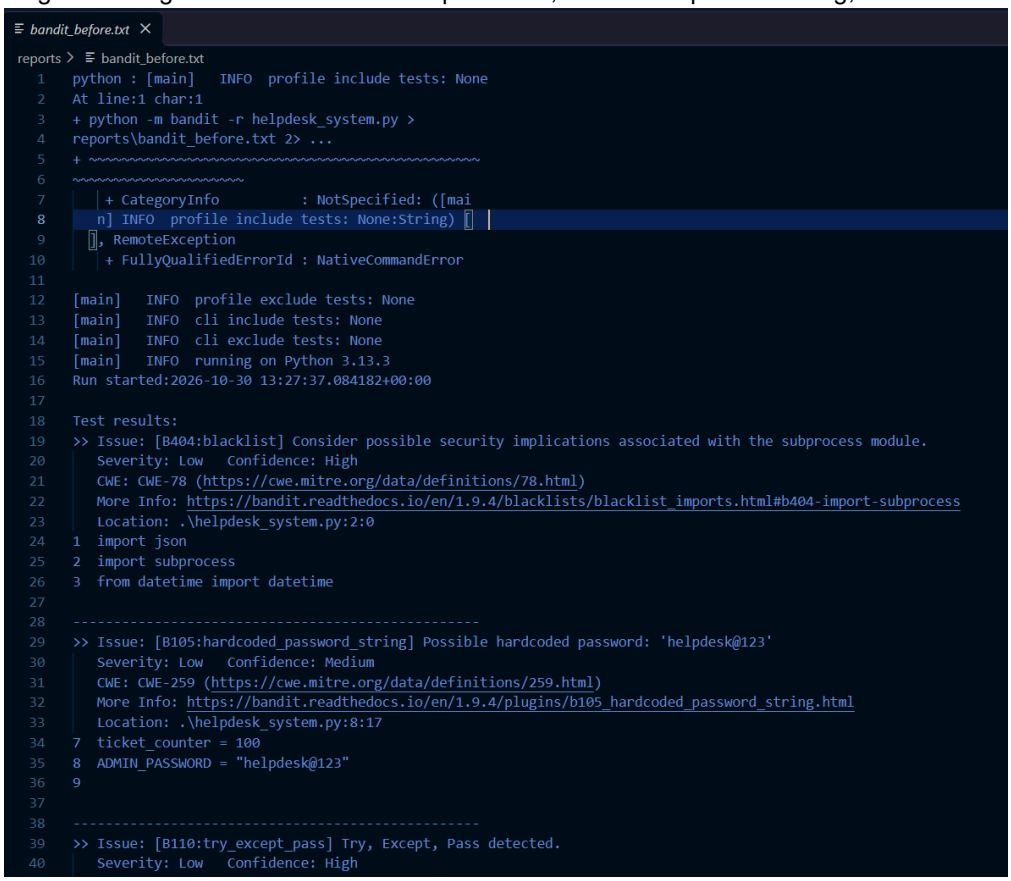
```
************* Module helpdesk_system
helpdesk_system.py:11:0: C0103: Constant name "ticket_counter" doesn't conform to UPPER_CASE naming style (invalid-name)
helpdesk_system.py:17:4: W0603: Using the global statement (global-statement)

--------------------------------------------------------------------
Your code has been rated at 9.67/10 (previous run: 9.67/10, +0.00)
```

4.3 Bandit - Before

File: reports/bandit_before.txt

Original findings included a hardcoded password, broad exception handling, and shell=True subprocess use.



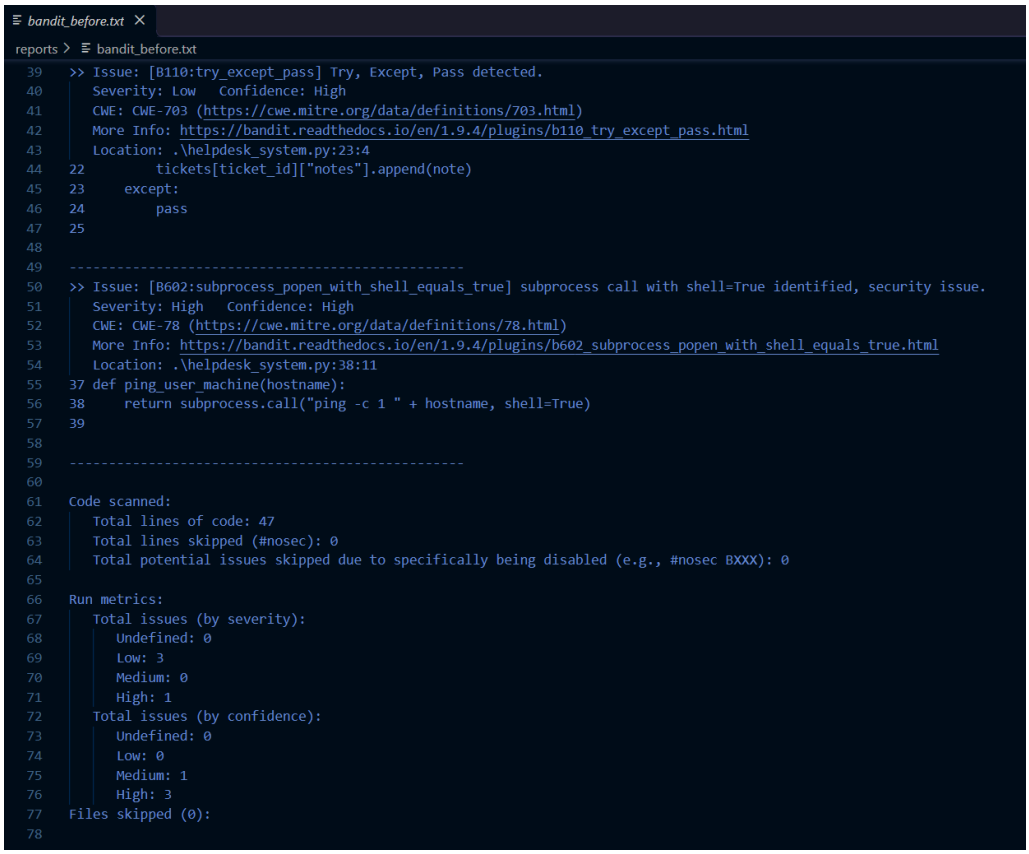
```
bandit_before.txt
reports > bandit_before.txt
python : [main]   INFO  profile include tests: None
At line:1 char:1
+ python -m bandit -r helpdesk_system.py >
reports\bandit_before.txt 2> ...
+ ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : NotSpecified: ([mai
   n] INFO  profile include tests: None:String) [ |
  ], RemoteException
    + FullyQualifiedErrorId : NativeCommandError

[main]   INFO  profile exclude tests: None
[main]   INFO  cli include tests: None
[main]   INFO  cli exclude tests: None
[main]   INFO  running on Python 3.13.3
Run started:2026-10-30 13:27:37.084182+00:00

Test results:
>> Issue: [B404:blacklist] Consider possible security implications associated with the subprocess module.
   Severity: Low   Confidence: High
   CWE: CWE-78 (https://cwe.mitre.org/data/definitions/78.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/blacklists/blacklist_imports.html#b404-import-subprocess
   Location: .\helpdesk_system.py:2:0
1  import json
2  import subprocess
3  from datetime import datetime

--------------------------------------------------
>> Issue: [B105:hardcoded_password_string] Possible hardcoded password: 'helpdesk@123'
   Severity: Low   Confidence: Medium
   CWE: CWE-259 (https://cwe.mitre.org/data/definitions/259.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b105_hardcoded_password_string.html
   Location: .\helpdesk_system.py:8:17
7  ticket_counter = 100
8  ADMIN_PASSWORD = "helpdesk@123"
9

--------------------------------------------------
>> Issue: [B110:try_except_pass] Try, Except, Pass detected.
   Severity: Low   Confidence: High
```



```
bandit_before.txt
reports > bandit_before.txt
>> Issue: [B110:try_except_pass] Try, Except, Pass detected.
   Severity: Low   Confidence: High
   CWE: CWE-703 (https://cwe.mitre.org/data/definitions/703.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b110_try_except_pass.html
   Location: .\helpdesk_system.py:23:4
22          tickets[ticket_id]["notes"].append(note)
23      except:
24          pass
25

--------------------------------------------------
>> Issue: [B602:subprocess_popen_with_shell_equals_true] subprocess call with shell=True identified, security issue.
   Severity: High   Confidence: High
   CWE: CWE-78 (https://cwe.mitre.org/data/definitions/78.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b602_subprocess_popen_with_shell_equals_true.html
   Location: .\helpdesk_system.py:38:11
37 def ping_user_machine(hostname):
38      return subprocess.call("ping -c 1 " + hostname, shell=True)
39

--------------------------------------------------

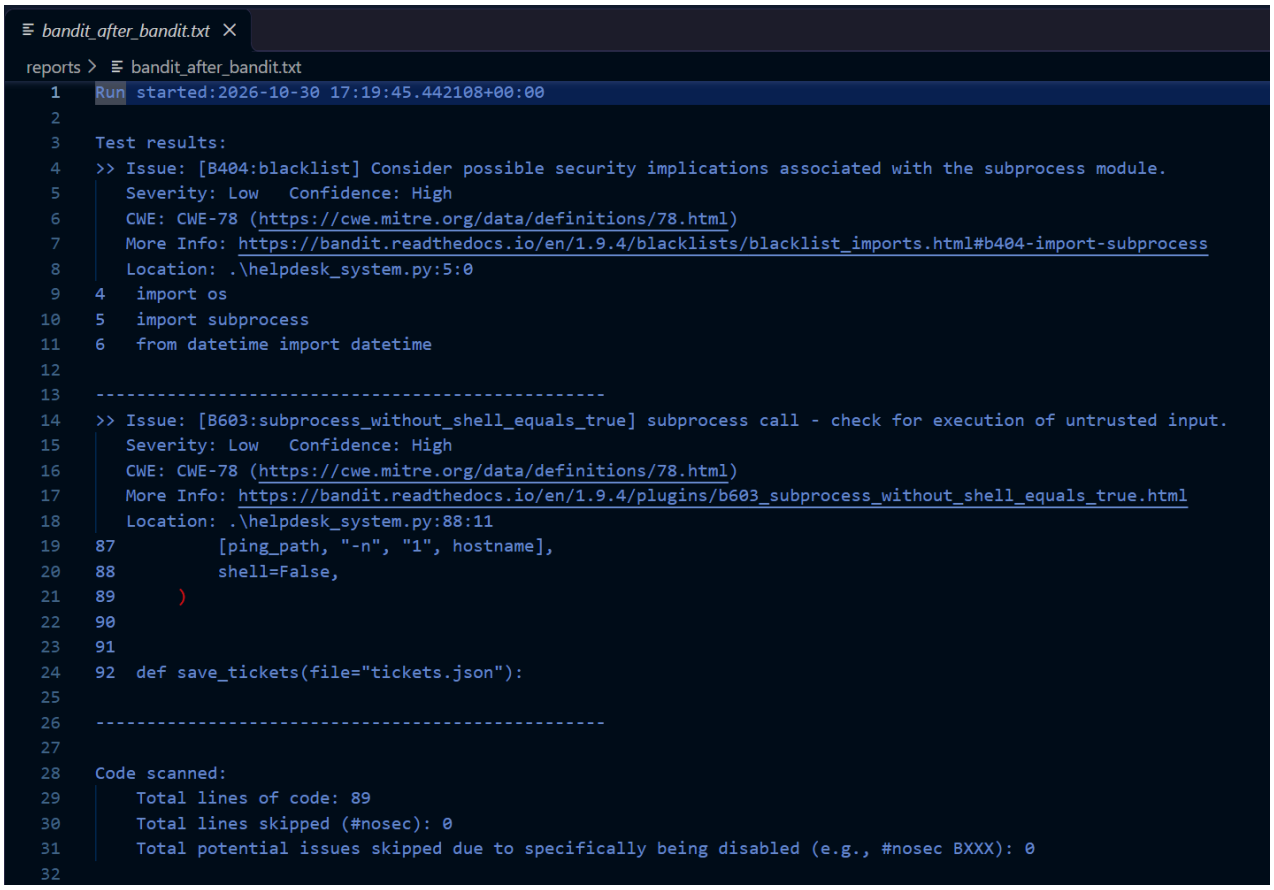
Code scanned:
   Total lines of code: 47
   Total lines skipped (#nosec): 0
   Total potential issues skipped due to specifically being disabled (e.g., #nosec BXXX): 0

Run metrics:
   Total issues (by severity):
      Undefined: 0
      Low: 3
      Medium: 0
      High: 1
   Total issues (by confidence):
      Undefined: 0
      Low: 0
      Medium: 1
      High: 3
Files skipped (0):
```

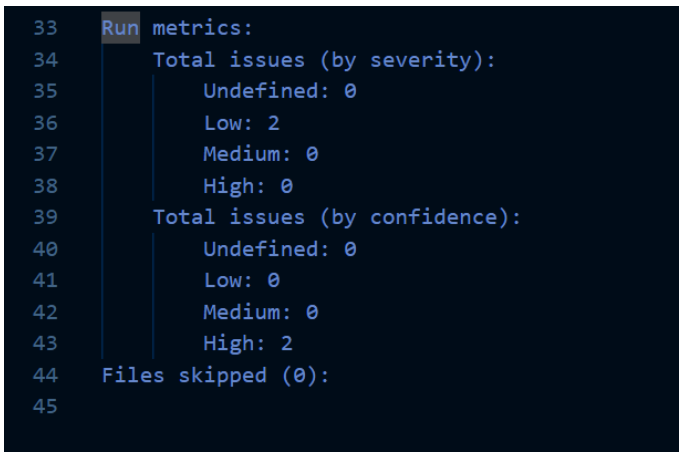
4.4 Bandit - After

File: reports/bandit_after_bandit.txt

Two low-severity findings remain: B404 (subprocess import) & B603 (subprocess call). The hardcoded password & shell=True findings were removed. No high-severity findings remain.



```
bandit_after_bandit.txt
reports > bandit_after_bandit.txt
1   Run started:2026-10-30 17:19:45.442108+00:00
2
3   Test results:
4   >> Issue: [B404:blacklist] Consider possible security implications associated with the subprocess module.
5      Severity: Low   Confidence: High
6      CWE: CWE-78 (https://cwe.mitre.org/data/definitions/78.html)
7      More Info: https://bandit.readthedocs.io/en/1.9.4/blacklists/blacklist_imports.html#b404-import-subprocess
8      Location: .\helpdesk_system.py:5:0
9   4   import os
10  5   import subprocess
11  6   from datetime import datetime
12
13  --------------------------------------------------
14  >> Issue: [B603:subprocess_without_shell_equals_true] subprocess call - check for execution of untrusted input.
15     Severity: Low   Confidence: High
16     CWE: CWE-78 (https://cwe.mitre.org/data/definitions/78.html)
17     More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b603_subprocess_without_shell_equals_true.html
18     Location: .\helpdesk_system.py:88:11
19  87          [ping_path, "-n", "1", hostname],
20  88          shell=False,
21  89      )
22  90
23  91
24  92  def save_tickets(file="tickets.json"):
25
26  --------------------------------------------------
27
28  Code scanned:
29      Total lines of code: 89
30      Total lines skipped (#nosec): 0
31      Total potential issues skipped due to specifically being disabled (e.g., #nosec BXXX): 0
32
```



```
33  Run metrics:
34      Total issues (by severity):
35          Undefined: 0
36          Low: 2
37          Medium: 0
38          High: 0
39      Total issues (by confidence):
40          Undefined: 0
41          Low: 0
42          Medium: 0
43          High: 2
44  Files skipped (0):
45
```

4.5 Flake8 - Before

File: reports/flake8_before.txt

The original report listed formatting/style issues including long lines, blank-line violations, bare except, comparison with None & an unused variable.

flake8_before.txt

reports > flake8_before.txt

```
helpdesk_system.py:10:1: E302 expected 2 blank lines, found 1
helpdesk_system.py:13:80: E501 line too long (118 > 79 characters)
helpdesk_system.py:14:80: E501 line too long (86 > 79 characters)
helpdesk_system.py:17:1: E302 expected 2 blank lines, found 1
helpdesk_system.py:20:1: E302 expected 2 blank lines, found 1
helpdesk_system.py:23:5: E722 do not use bare 'except'
helpdesk_system.py:26:1: E302 expected 2 blank lines, found 1
helpdesk_system.py:30:1: E302 expected 2 blank lines, found 1
helpdesk_system.py:33:63: E711 comparison to None should be 'if cond is None:'
helpdesk_system.py:37:1: E302 expected 2 blank lines, found 1
helpdesk_system.py:40:1: E302 expected 2 blank lines, found 1
helpdesk_system.py:45:1: E302 expected 2 blank lines, found 1
helpdesk_system.py:47:5: F841 local variable 't2' is assigned to but never used
helpdesk_system.py:52:80: E501 line too long (80 > 79 characters)
helpdesk_system.py:58:1: E305 expected 2 blank lines after class or function definition, found 1
```

4.6 Flake8 - After

File: reports/flake_after.txt

The file is empty (0 bytes) because Flake8 produced no diagnostics after the fixes.

flake8_after.txt

reports > flake8_after.txt

```
1
```

5. After-Analysis Terminal Evidence

```
(.venv) PS C:\Users\rdhor\Desktop\SE-Labs-PES1UG24CS370\Lab-5> python -m pylint helpdesk_system.py --output-format=text > reports/pylint_after_pylint.txt

(.venv) PS C:\Users\rdhor\Desktop\SE-Labs-PES1UG24CS370\Lab-5> Get-Content reports/pylint_after_pylint.txt
************* Module helpdesk_system
helpdesk_system.py:11:0: C0103: Constant name "ticket_counter" doesn't conform to UPPER_CASE naming style (invalid-name)
helpdesk_system.py:17:4: W0603: Using the global statement (global-statement)

-------------------------------------------------------------------
Your code has been rated at 9.67/10 (previous run: 9.67/10, +0.00)
```

```
(.venv) PS C:\Users\rdhor\Desktop\SE-Labs-PES1UG24CS370\Lab-5> python -m bandit helpdesk_system.py -f txt -o reports/bandit_after_bandit.txt
[main]  INFO    profile include tests: None
[main]  INFO    profile exclude tests: None
[main]  INFO    cli include tests: None
[main]  INFO    cli exclude tests: None
[main]  INFO    running on Python 3.13.3
[text]  INFO    Text output written to file: reports/bandit_after_bandit.txt
```

```
(.venv) PS C:\Users\rdhor\Desktop\SE-Labs-PES1UG24CS370\Lab-5> Get-Content reports/bandit_after_bandit.txt
Run started:2026-10-30 17:19:45.442108+00:00

Test results:
>> Issue: [B404:blacklist] Consider possible security implications associated with the subprocess module.
   Severity: Low   Confidence: High
   CWE: CWE-78 (https://cwe.mitre.org/data/definitions/78.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/blacklists/blacklist_imports.html#b404-import-subprocess
   Location: .\helpdesk_system.py:5:0
4       import os
5       import subprocess
6       from datetime import datetime

--------------------------------------------------
>> Issue: [B603:subprocess_without_shell_equals_true] subprocess call - check for execution of untrusted input.
   Severity: Low   Confidence: High
   CWE: CWE-78 (https://cwe.mitre.org/data/definitions/78.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b603_subprocess_without_shell_equals_true.html
   Location: .\helpdesk_system.py:88:11
87              [ping_path, "-n", "1", hostname],
88              shell=False,
89          )
90
91
92      def save_tickets(file="tickets.json"):

--------------------------------------------------

Code scanned:
        Total lines of code: 89
        Total lines skipped (#nosec): 0
        Total potential issues skipped due to specifically being disabled (e.g., #nosec BXXX): 0
```

```
Run metrics:
        Total issues (by severity):
                Undefined: 0
                Low: 2
                Medium: 0
                High: 0
        Total issues (by confidence):
                Undefined: 0
                Low: 0
                Medium: 0
                High: 2
Files skipped (0):
```

```
(.venv) PS C:\Users\rdhor\Desktop\SE-Labs-PES1UG24CS370\Lab-5> python -m flake8 helpdesk_system.py --output-file=reports/flake_after.txt
(.venv) PS C:\Users\rdhor\Desktop\SE-Labs-PES1UG24CS370\Lab-5> Get-Item reports/flake_after.txt | Select-Object Name, Length

Name           Length
----           ------
flake_after.txt      0
```

6. Git Commit Evidence

The code changes were committed separately for the three tools and pushed to the main branch.

Tool	Commit	Commit message
Pylint	5eeb3f2	fix: resolve Pylint code quality issues
Bandit	8116705	fix: address Bandit security findings
Flake8	2953020	style: resolve Flake8 formatting issues
Bandit	8521ac0	fix: harden ping command validation
Docs	9eb9471	docs: update static analysis reports
Docs	d213869	docs: remove duplicate Bandit report



7. Reflection

What did you learn from Pylint?

Pylint helped identify maintainability and code-quality issues such as missing documentation, mutable default arguments, naming conventions, and unused or unclear constructs. The score improved from 5.53/10 to 9.67/10. The remaining warnings are documented rather than hidden.

What did you learn from Bandit?
Bandit highlighted potentially risky patterns, including a hardcoded password and shell=True subprocess use. The password was moved to an environment variable and subprocess was changed to use an argument list with shell=False. Two low-severity subprocess-related findings remain and should be reviewed in context.

What did you learn from Flake8?
Flake8 made formatting and style problems visible, including line length, blank lines, exception syntax, comparison with None, and an unused variable. After corrections, Flake8 produced no diagnostics.

How does static analysis improve software quality?
Static analysis gives fast, repeatable feedback before code is executed. Using Pylint, Bandit, and Flake8 together provides complementary checks for code quality, security patterns, and style. Tool findings still require developer judgment; a clean report does not prove that a program is completely secure or bug-free.

8. Conclusion

The helpdesk program was updated after analysis with Pylint, Bandit, and Flake8. Pylint improved from 5.53/10 to 9.67/10, Flake8 reported no remaining style diagnostics, and Bandit no longer reported the hardcoded password or shell=True findings. Two low-severity subprocess-related Bandit findings remain documented. The changes were committed separately for the three tools.